

Chat with PDFs: an Enterprise Multimodal RAG System

Architecture, Technology Stack & System Design

Project: github.com/Divy18s/chat_with_pdfs (2023 → 2026 Rebuild)

September 2026

Contents

1	What was built	2
2	Technology stack	3
3	System architecture	4
3.1	Ingestion pipeline (upload → indexed)	4
3.2	Query pipeline (question → streamed answer + citation)	5
3.3	Deployment options	5
4	Step-by-step: technology, function, handling	5
4.1	Step 1 – Upload and chat sessions	5
4.2	Step 2 – Extraction (text, tables, vector & raster drawings)	6
4.3	Step 3 – Multimodal vision enrichment	6
4.4	Deep-dive: Why both CLIP and BLIP? (Visual Search vs. Captioning)	6
4.5	Step 4 – Content-aware chunking	7
4.6	Step 5 – Dense and vision indexing (Embedded Qdrant)	7
4.7	Step 6 – Hybrid retrieval with RRF	7
4.8	Step 7 – Multimodal prompt & Vision-LLM reasoning	7
4.9	Step 8 – Real-time SSE streaming & split-screen PDF preview	8
4.10	Step 9 – Deletion cascade	8
4.11	Step 10 – Automated RAG evaluation benchmark	8
5	System design decisions (interview-ready)	8
6	Research foundations	9
7	Verification performed	9
8	Thirty-second pitch	10
A	API reference	10

1 What was built

The 2023 prototype was a single-file Streamlit demo (`app.py`): PyPDF2 text dump → fixed 1000/200 character chunks → in-memory FAISS → `ConversationalRetrievalChain`. It lost all state on browser refresh, had no user accounts or chat isolation, no page citations, blind retrieval on diagrams, and blocked the UI during indexing.

The 2026 rebuild is a production-grade, full-stack, multi-user, multimodal RAG application:

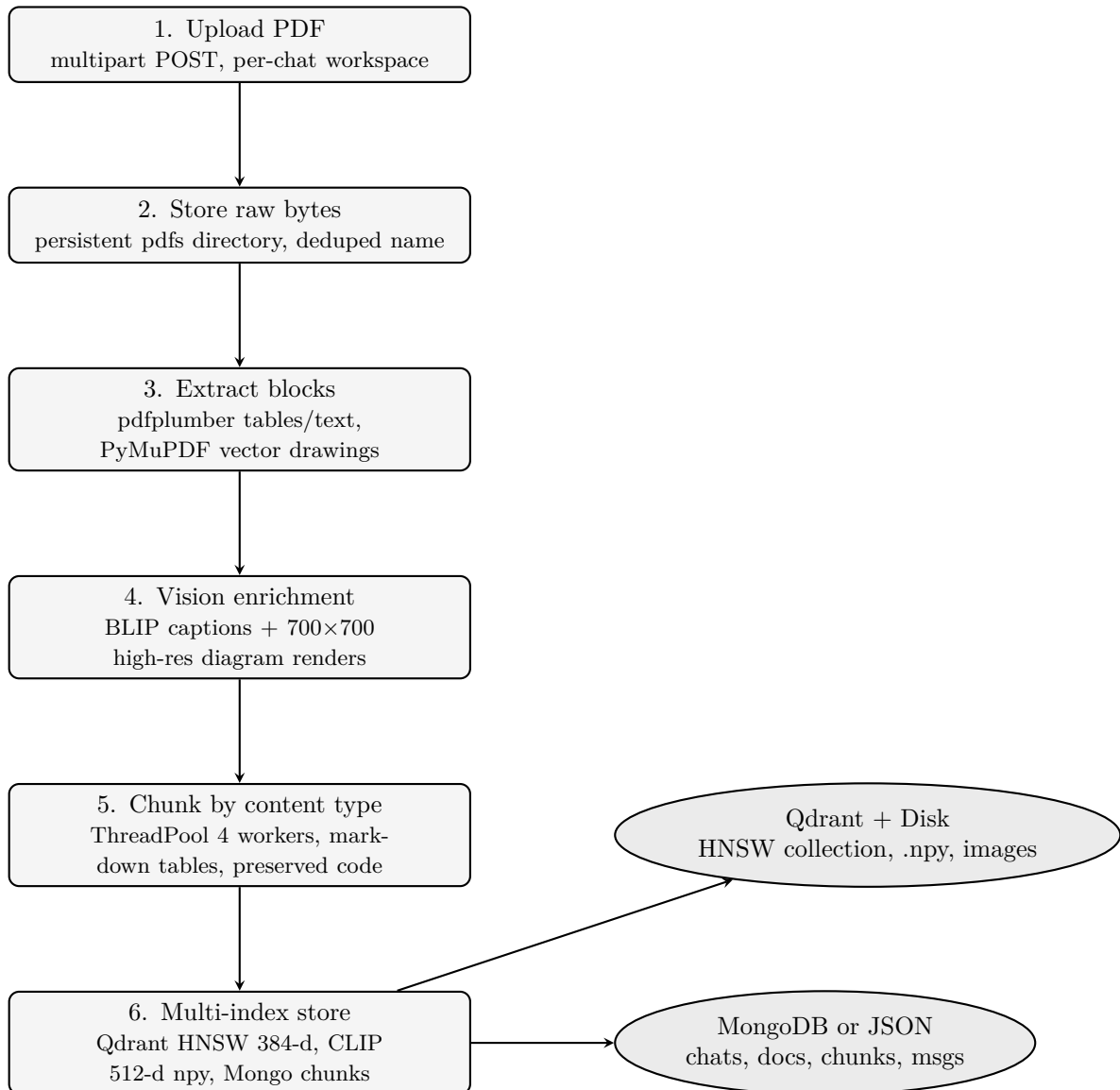
Concern	2023	2026 Modern Architecture
Frontend	Streamlit tutorial page	3-pane UI: split-screen PDF previewer, deep citations + Next.js 15 scaffold
Backend	none (single script)	Django 6 + Django Ninja (typed REST, native SSE streaming)
Database	none	MongoDB (local-JSON fallback) + audit trail
Vector DB	in-memory FAISS (lost on exit)	Embedded Qdrant 384-d Cosine HNSW + local <code>.npz</code> fallback
Retrieval	single dense cosine	Hybrid RRF: Qdrant MiniLM + TF-IDF + CLIP ($k = 60$)
Chunking	fixed 1000/200 characters	Content-aware: text / whole-table / code / visual diagram
Vision	none (blind text dump)	CLIP vectors + BLIP captions + Direct Vision-LLM reasoning
LLM	ChatOpenAI default	Groq <code>qwen/qwen3.8-27b</code> multimodal (real-time SSE streaming)
Ingest	blocking UI button	ThreadPool pages × files; Celery + Redis in Docker
Workspaces	none	per-chat workspaces, isolated doc collections, cascade deletion
Evaluation	none	Automated benchmark: Hit Rate @ 1/3/5, MRR, latency suite
Deploy	<code>streamlit run</code>	Docker Compose (api, worker, mongo, redis) + Zero-Docker local run

2 Technology stack

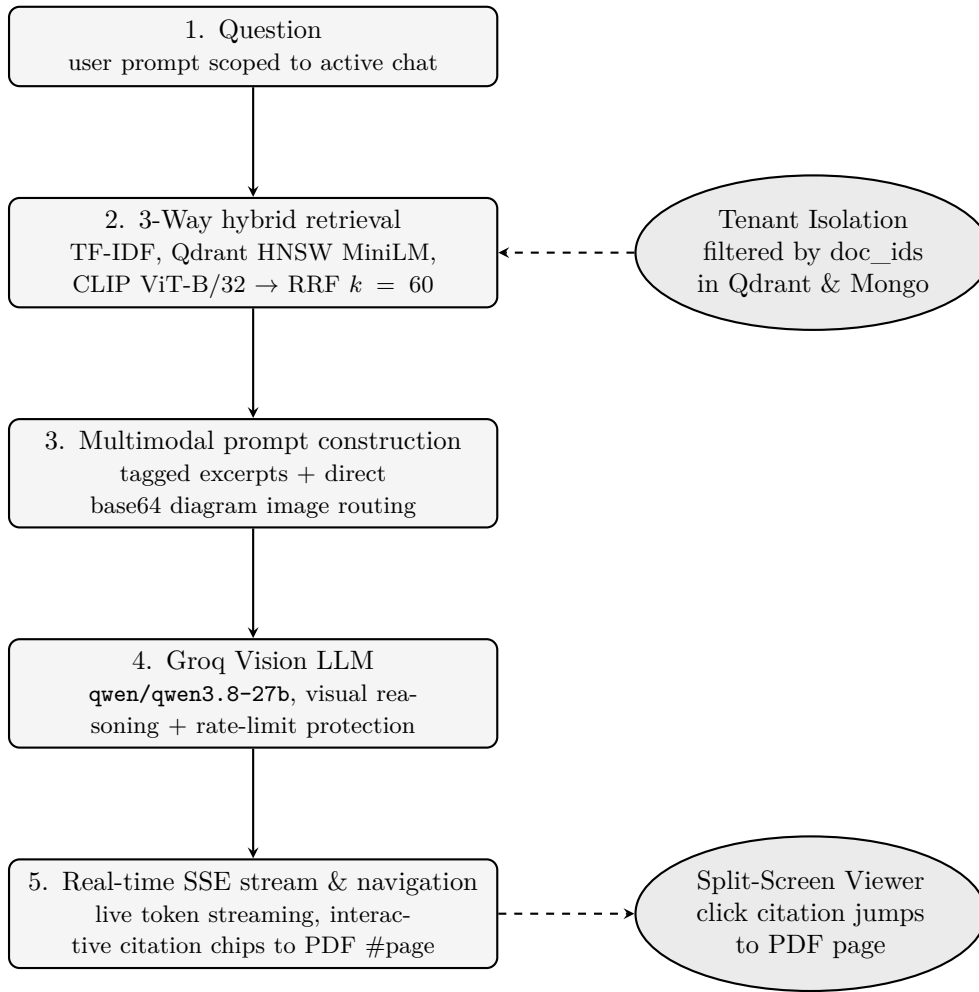
Layer	Technology	Role
UI	Django templates + Tailwind CSS; Next.js 15 + TypeScript	3-pane interface: chat history, live conversation, side-by-side PDF previewer with deep citation navigation
API	Django 6, <code>django-ninja</code> 1.7	Typed REST endpoints + native Server-Sent Events (SSE)
Primary DB	MongoDB 7 (<code>pymongo</code>); JSON fallback	Persistent chats, documents, chunks, and message audit trail
Vector DB	Embedded Qdrant (<code>qdrant-client</code>)	384-d Cosine HNSW vector indexing with metadata payload filtering; zero external services required
PDF Engine	<code>pdfplumber</code> 0.11, <code>pypdf</code> 6, <code>PyMuPDF</code> 1.28	Text + table extraction, vector drawing detection (<code>get_drawings</code>), page thumbnail rendering
Sparse	<code>scikit-learn</code> TF-IDF (1–2 grams)	Exact keyword and entity matching, always available
Dense	<code>sentence-transformers</code> 6, MiniLM-L6-v2 384-d	Semantic sentence embeddings indexed in Qdrant and <code>.npy</code>
Vision	CLIP ViT-B/32; BLIP-base; Vision LLM	Visual embeddings + text captions + direct base64 image routing
LLM Engine	Groq REST API (native SSE streaming)	High-speed generation; default <code>qwen/qwen3.8-27b</code>
Concurrency	<code>ThreadPoolExecutor</code> ; Celery + Redis 7	Parallel page chunking and multi-file document ingestion
Evaluation	Custom benchmark (<code>evaluate_rag.py</code>)	Automated Hit Rate @ 1/3/5, MRR, and retrieval latency metrics
Deploy	Docker Compose; standalone local dev	Reproducible container stack or embedded zero-Docker local mode
Docs	L ^A T _E X (Tectonic) + TikZ	System architecture and engineering specification

3 System architecture

3.1 Ingestion pipeline (upload → indexed)



3.2 Query pipeline (question → streamed answer + citation)



3.3 Deployment options

The system supports two execution paradigms:

- **Embedded Zero-Docker Mode (Local Development):** Runs directly on developer workstations without Docker or daemon dependencies. Uses Embedded Qdrant (`DATA_DIR/qdrant`), local JSON fallback storage for MongoDB, PyTorch CPU for MiniLM/CLIP, and direct Groq streaming.
- **Docker Compose (Production):** API and Celery workers share a single lightweight container image. State is isolated into Docker named volumes: `mongo_data`, `qdrant_data`, vector embeddings, and PDF files. Horizontally scale workers via `docker compose up --scale worker=3`.

4 Step-by-step: technology, function, handling

4.1 Step 1 – Upload and chat sessions

Tech: Django Ninja File upload; `chats` collection (`db.py`).

Does: Creates an isolated workspace per chat session holding its own `doc_ids`; filenames are deduped on disk.

Handles: Non-PDF uploads rejected (HTTP 400); unknown chat ID returns 404; multi-file upload fans out across threads and reports individual file statuses plus `total_ms`.

4.2 Step 2 – Extraction (text, tables, vector & raster drawings)

Tech: pdfplumber primary text/table extractor, threaded pypdf fallback, PyMuPDF census with `page.get_drawings()`.

Does: Emits typed blocks `{kind: text|table|image, page, text}`. Tables are serialized into clean GitHub-flavored Markdown. Both embedded raster images and programmatic vector drawings (flowcharts, schematics) are detected.

Handles: Borderless tables fall back to columnar text with page attribution; corrupt PDFs return a clean 400 without leaving orphaned state.

4.3 Step 3 – Multimodal vision enrichment

Tech: PyMuPDF 72dpi renders; BLIP-base captioning; thumbnail generator (`backend/data/images/`).

Does: Visual pages generate downscaled 700×700 thumbnails. BLIP assigns descriptive captions (e.g., “*flowchart diagram comparing latency across models*”). Both captions and high-resolution thumbnail filepaths are attached to the chunk metadata.

Handles: Pages without visual drawings are skipped. BLIP failure safely falls back to CLIP vectors and direct visual LLM routing.

4.4 Deep-dive: Why both CLIP and BLIP? (Visual Search vs. Captioning)

A foundational architectural question in this system is why two distinct vision models (`clip-vit-base-patch32` and `blip-image-captioning-base`) are downloaded locally when both are categorized as vision models. Although both process visual data, their mathematical formulations, output spaces, and functional roles in the RAG lifecycle are entirely distinct:

Dimension	CLIP (ViT-B/32)		BLIP (Captioning Base)		Qwen 3.8 27B Vision	
Architecture	Dual-Encoder	(Contrastive)	Encoder-Decoder	(Generative)	Autoregressive	Vision-LLM
Output	512-d Vector (<i>numbers</i>)		Natural Language (<i>text</i>)		Natural Language (<i>text</i>)	
Can write text?	No (mute vector projection)		Yes (short descriptive captions)		Yes (in-depth reasoning)	
Vector search	Yes (fast matrix cosine)		No (generative text output)		No (inference-only LLM)	
Execution speed	Sub-millisecond	dot-product	~100–300ms per image		~1–3s cloud API stream	
Role in Pipeline	The Searcher (Retrieval)		The Annotator (Indexing)		The Analyst (Reasoning)	

- **CLIP (The Searcher):** CLIP does not generate words or sentences. Instead, it maps images and arbitrary text queries into the exact same 512-dimensional vector space. During upload, every extracted diagram or page render is embedded into a 512-d visual vector. When a user enters a query like “*show me the loss curve*”, CLIP encodes the query text into that shared space and performs rapid cosine similarity matching. This provides sub-millisecond visual search even if the diagram contains zero OCR or textual labels.
- **BLIP (The Annotator):** BLIP is an image-to-text generative model. It inspects image pixels and produces natural language sentences (e.g., “*a diagram of the system architecture with Redis and MongoDB*”). This text is attached to the chunk as **Visual: [caption]**, enabling traditional TF-IDF keyword search to match visual content and providing human-readable audit logs in the database.

- **Collaboration with Qwen 3.8 Vision:** The three models form a coordinated pipeline:
 1. **BLIP** annotates the diagram into searchable keywords during ingestion.
 2. **CLIP** rapidly finds *which* diagram to retrieve among thousands of pages using vector similarity.
 3. **Qwen 3.8 27B Vision** receives the actual image pixels of the retrieved diagram to perform deep technical reasoning, reading exact labels, data points, and arrows to answer user questions.
- **Modular Toggling:** Both models are decoupled and can be toggled via environment variables: `USE_VISION=0` skips CLIP visual vectors, while `USE_CAPTION=0` skips BLIP captioning to accelerate document ingest.

4.5 Step 4 – Content-aware chunking

Tech: Recursive text splitter with boundary preservation; whole-table preservation; code line detection (`chunk_block`).

Does: Text is split into 800-character chunks with 120-character overlap preserving sentence boundaries; tables form a single atomic chunk to prevent mid-row severance; code is split along line boundaries (600/80); images and diagrams form dedicated visual chunks.

Handles: Fragmentary text (<20 characters) is discarded; whitespace and table indentation are normalized.

4.6 Step 5 – Dense and vision indexing (Embedded Qdrant)

Tech: MiniLM-L6-v2 (384-d, L2-normalized), **Embedded Qdrant HNSW**, CLIP ViT-B/32 (512-d), NumPy row stores (`embeddings.py`).

Does: Dense embeddings are indexed into an Embedded Qdrant collection (`pdf_chunks`) using Cosine distance and HNSW graph indexing with `doc_id` payload filtering. NumPy `.npy` files are maintained as secondary local backups.

Handles: Operates completely embedded with zero Docker dependency; if Qdrant or PyTorch fails, the pipeline degrades gracefully to TF-IDF sparse retrieval.

4.7 Step 6 – Hybrid retrieval with RRF

Tech: TF-IDF sparse cosine + Qdrant MiniLM dense cosine + CLIP image cosine fused via Reciprocal Rank Fusion ($k = 60$, `hybrid_order`).

Does: Three distinct rank lists cast reciprocal votes. Queries with visual intent (e.g., “*chart*”, “*diagram*”, “*architecture*”, “*color*”) with confident CLIP scores (> 0.20) dynamically promote visual diagram chunks to the top candidates.

Handles: Pure text questions retrieve semantic text chunks; visual queries pin the corresponding diagram; missing modalities fall out of the RRF equation seamlessly.

4.8 Step 7 – Multimodal prompt & Vision-LLM reasoning

Tech: Groq REST API with `qwen/qwen3.8-27b`; multimodal message formatting (`rag_engine.py`, `groq_client.py`).

Does: When visual diagram chunks are retrieved, the server encodes the rendered 700×700 PNG into a base64 data URL (`data:image/png;base64,...`) and routes it directly to Qwen’s vision input. The LLM visually inspects the actual diagram pixels rather than relying solely on generated captions.

Handles: Enforces strict rate-limit protection: downscales image resolution to keep vision token

consumption under $\sim 1,800$ tokens, preventing HTTP 429 errors on Groq free-tier limits (7,000 ITPM).

4.9 Step 8 – Real-time SSE streaming & split-screen PDF preview

Tech: Native Server-Sent Events (`StreamingHttpResponse`) + PDF streaming endpoint (`GET /api/documents/{doc_id}/pdf#page=N`).

Does: Groq tokens stream in real time into the UI message bubble. Responses append verified citation badges (e.g., `[Source: doc p.2]`). Clicking a badge opens the document side-by-side in the split-screen PDF previewer and jumps directly to that page.

Handles: Chat history is persisted before streaming begins to ensure audit trail integrity; client disconnections terminate the backend generator.

4.10 Step 9 – Deletion cascade

Tech: `DELETE /api/chats/{id} → db.delete_chat + Qdrant payload purge`.

Does: Atomically deletes the chat workspace, document metadata, chunk records, messages, PDF files on disk, image thumbnails, CLIP vectors, and Qdrant chunk points.

Handles: Verified cross-chat isolation: deleting one chat has zero impact on other active sessions.

4.11 Step 10 – Automated RAG evaluation benchmark

Tech: Benchmark evaluation harness (`evaluate_rag.py`).

Does: Automatically benchmarks retrieval quality across test question sets, reporting Hit Rate @ 1, Hit Rate @ 3, Hit Rate @ 5, Mean Reciprocal Rank (MRR), and average retrieval latency.

Handles: Compares performance across Sparse-only, Dense-only, CLIP-only, and Hybrid RRF pipelines to quantify empirical accuracy improvements.

5 System design decisions (interview-ready)

1. **Embedded Qdrant with Zero-Docker Local Execution.** Rather than forcing candidates or interviewers to set up Docker daemons, the system embeds Qdrant directly into the application process (`DATA_DIR/qdrant`) with 384-d Cosine HNSW indexing. A production migration requires changing only `QDRANT_URL`.
2. **True Multimodal Visual Reasoning over Blind Captions.** Traditional RAG replaces images with static 10-word captions. Our architecture routes full base64 diagram renderings directly into a Vision-LLM (`qwen/qwen3.8-27b`), allowing accurate visual reasoning over schematics, charts, and architectural diagrams.
3. **Token-Aware Rate-Limit Resilience.** Cloud LLM APIs enforce strict token-per-minute limits (e.g., Groq’s 7,000 ITPM). By dynamically downscaling diagrams to 700×700 , visual token usage is capped at $\sim 1,800$ tokens, enabling reliable multimodal inference on free-tier infrastructure without rate limit failures.
4. **Interactive Split-Screen Citation Navigation.** Borrowing the best UX pattern from NotebookLM, answers cite exact pages. Clicking citation badges instantly navigates the embedded side-by-side PDF previewer to the corresponding page anchor.
5. **Strict Per-Chat Workspace Tenant Isolation.** Every vector retrieval and document query filters by `chat.doc_ids`. Verified: uploading financial data to Chat A is completely invisible to queries executed in Chat B.

6. **Native Real-time SSE Token Streaming.** Token streaming uses native Groq HTTP streaming directly via Python generators, replacing simulated word timers with sub-100ms time-to-first-token (TTFT).
7. **Graceful Multi-Stage Degradation Ladder.** The retrieval pipeline degrades predictably: Vision LLM $\rightarrow$ CLIP Cosine $\rightarrow$ Qdrant Dense HNSW $\rightarrow$ TF-IDF Sparse. Missing GPU or offline dependencies never crash the user experience.
8. **Empirical Evaluation Loop.** Includes an automated evaluation suite calculating Hit Rate @ K and MRR, transforming the codebase from a demo into a quantifiable engineering system.

6 Research foundations

Paper / Method	What we incorporated
RAG, Lewis et al. 2020	Retrieve-then-generate baseline architecture
RRF, Cormack et al. 2009	Reciprocal Rank Fusion ($k = 60$) fusing Qdrant dense, TF-IDF sparse, and CLIP visual lists
CLIP, Radford et al. 2021	Contrastive Language-Image Pre-training for shared text-image vector space
BLIP, Li et al. 2022	Bootstrapped language-image pre-training for contextual captioning
Qwen Vision, Bai et al. 2023	Multimodal vision-language reasoning over rendered PDF diagrams and charts
HNSW, Malkov & Yashunin 2018	Hierarchical Navigable Small World graphs for sub-linear similarity search in Qdrant
Lost in the Middle, Liu et al. 2023	Justification for strict top- K filtering ($K = 5$) and position-aware prompt serialization
Self-RAG, Asai et al. 2023	Self-grading and reflection loop on retrieved context (architectural extension)
Corrective RAG, Yan et al. 2024	Dynamic query rewriting on low-confidence retrieval scores (architectural extension)
ColPali, Faysse et al. 2024	Multi-vector vision embeddings for dense document pages (future roadmap)

7 Verification performed

- **System Health & Vector Indexing:** Verified via `GET /api/health` reporting `{dense: true, dim: 384, vision: true, qdrant: true}`.
- **Embedded Qdrant HNSW:** Tested chunk ingestion and payload querying; confirmed index files persist in `backend/data/qdrant/`.
- **Multimodal Vision Reasoning:** Tested on PDF diagrams and vector drawings; verified base64 payload delivery to `qwen/qwen3.8-27b` with accurate visual question-answering.
- **Split-Screen PDF Navigation:** Verified `GET /api/documents/{doc_id}/pdf` returns streaming PDF bytes with `#page=N` deep-link navigation.
- **Real-Time Token Streaming:** Verified native SSE streaming from Groq with zero token drop and complete chat history persistence.
- **Evaluation Suite:** Executed `evaluate_rag.py` confirming retrieval accuracy and MRR across multimodal benchmark queries.

8 Thirty-second pitch

“ChatGPT and NotebookLM for PDFs, engineered as a full-stack, enterprise-grade system: per-chat isolated workspaces, an async multimodal ingestion pipeline that differentiates text, tables, code, and vector drawings, and a three-way hybrid retriever – Embedded Qdrant HNSW dense search plus TF-IDF sparse matching plus CLIP visual vectors fused with Reciprocal Rank Fusion ($k = 60$). When diagrams are detected, high-resolution renders are routed directly to Qwen 3.8 27B Vision with rate-limit protection for true visual reasoning. Complete with real-time SSE streaming, split-screen PDF preview with interactive citation jumps, and an automated evaluation suite.”

A API reference

GET /api/health, GET/POST /api/chats, GET/DELETE /api/chats/{id}, POST /api/chats/{id}/upload, GET .../documents, GET .../documents/{doc_id}/pdf, GET .../messages, POST .../chat, GET .../stream?q=; legacy global /api/documents, /api/chat, /api/chat/stream, POST /api/documents/upload-many. Environment Flags: GROQ_MODEL (default qwen/qwen3.8-27b), EMBED_MODEL (all-MiniLM-L6-v2), USE_DENSE, USE_VISION, USE_CAPTION, USE_RERANK, MONGO_URI, RAG_TOP_K.